Contents

The Aegis: From Three Prompt Calls to a Multi-Provider Multi-Agent System **1**
Abstract 1
1. The Problem 2
2. Design Choices 2
3. How It Works 3
4. Trial and Error 6
5. Numbers 9
6. Reflection 12

The Aegis: From Three Prompt Calls to a Multi-Provider Multi-Agent System

Abdullah Hasan · Student ID: 807271 Source code: github.com/Abdullah-373/The-Aegis

---

Abstract

The Aegis is a small web app I built to help review contracts. The user uploads a PDF, several AI agents argue about it, and the app returns a structured verdict — GO, NO-GO, or CONDITIONAL-GO (rendered to users as **PROCEED**, **WALK AWAY**, and **MAYBE**) — together with a 0-to-100 risk score and a list of conditions to fix.

The project went through three iterations. **Version 1** was three sequential LLM calls in fixed roles: Alex argues the bullish case, Sam attacks it, Maya rules. It worked but felt thin — three prompt-chained calls is not really a multi-agent system. **Version 2** replaced the straight line with a LangGraph state machine: a Planner picks which specialist analysts to run, the specialists can call a `search_precedent` tool against a built-in knowledge base of 34 risk patterns, the Strategist and Red Team synthesise their findings, the Judge rules with a strict JSON schema, and a critique loop lets Alex and Sam respond to Maya. If either dissents, Maya runs once more. **Version 3** (the current build) adds a second model provider — OpenAI's GPT family — alongside Gemini, so the same pipeline can be driven by `gpt-5`, `gpt-5-mini`, `gpt-4o`, or `gpt-4o-mini` whenever the user prefers the OpenAI ecosystem or wants to escape Gemini's 20-requests-per-day free tier cap.

Both v1 and v2 ship in the same app behind a Fast / Full mode switch. Fast keeps the original three-call design (3 API requests per analysis); Full runs the multi-agent pipeline (8 to 15 requests). The toggle exists because the Gemini free tier is capped at 20 requests per day on `gemini-2.5-flash` — Fast gives you about six analyses per day on free tier, Full gives you one or two.

Measured against the live OpenAI and Gemini APIs across three sample contracts and four models:

- **Full multi-agent / GPT-5 on `contract_balanced.pdf`**: 617.03 s / 6,704 tokens / $0.0941 list-price / CONDITIONAL-GO with risk score 61.
- **Cache hit on the same row**: 1.68 s wall-clock replay; **saved 617.03 s** of compute and $0.0941 of API spend on the second click. The cache-hit measurement gap I flagged in the previous version of this report is now closed.

I ran every test against live APIs; the dollar figures are list-price equivalents at the providers' published per-token rates.

---

1. The Problem

Most business decisions involve documents that nobody really has time to read end to end. Contracts and service agreements often run for tens of pages. Either you skim them and miss the bad parts, or you pay a lawyer to read every page, which gets expensive fast.

Existing AI summarisers exist but they have two problems for this use case. The first is that a single AI tends to merge optimistic and pessimistic readings into one neutral summary. For a contract that neutral middle is exactly the wrong answer — what you want to know is the worst thing about the deal together with the best thing about it, both at full strength, so you can decide what to do. The second problem is that AI output is free-form text. If you want a yes/no decision out of it, you need to parse the prose with regular expressions, which breaks the first time the model phrases its answer slightly differently.

The Aegis tries to fix both. The system never gives you a neutral summary — instead it surfaces the bullish case and the bearish case as separate, traceable outputs and then a third agent reads both and rules with a structured JSON answer that downstream code can consume directly.

---

2. Design Choices

This section explains *why* I picked the building blocks I did, rather than listing them as a survey.

Adversarial roles, not one neutral summary. The single-summary approach hides the spread between optimism and pessimism. The literature on multi-agent debate (Du et al. 2023 and similar) shows that having two LLMs argue often catches mistakes that one model alone misses. I adapted this into a fixed Strategist → Red Team → Judge sequence rather than a free chat, because for contract review I wanted traceable transcripts that a reader can audit later, not a turn-by-turn dialogue.

LangGraph for the orchestration in v2. Once the second version grew past three sequential calls, hand-rolling the control flow in `main.py` was getting ugly. I moved the agent topology into a LangGraph `StateGraph` with a typed state object and explicit nodes. Conditional edges handle "if either critic dissents, send the ruling back to

Maya for revision." The graph also gives me a clean diagram I can hand to anyone reviewing the code.

Tool use with a tiny in-code knowledge base. Modern agent systems expose tools (function calls) to the LLM so the model can retrieve information rather than rely on what it memorised. I gave the specialists and Sam access to a single tool: `search_precedent(query)`, which retrieves the top-4 entries from a built-in corpus of 34 contract risk patterns. I implemented the retrieval as plain TF-IDF + cosine similarity in pure Python because the corpus is small enough that an external vector store would be over-engineering.

Pydantic for the final ruling. Maya is forced to end her response with a fenced JSON block matching a strict schema. Pydantic with `Literal` enums validates it. Three fallback layers handle the times the model gets the JSON slightly wrong.

WebSockets over Server-Sent Events. The connection is two-way: the same socket carries the API key, the PDF binary, and the streaming verdict back. No separate upload round-trip.

SQLite with a content-hash key, not Postgres or Redis. The cache key is `SHA-256(extracted_text + model_name)`, so renamed copies of the same PDF hit the cache, but model upgrades correctly invalidate prior rulings. SQLite needs zero infrastructure and is fast enough for single-user scenarios.

Multi-provider in v3. I added OpenAI alongside Gemini because (a) the Gemini free-tier cap kept blocking my own testing in Full mode, and (b) reviewers wanted to see how the same pipeline behaves under a different model family. The provider is detected from the key prefix (`AIza…` → Google, `sk-…` → OpenAI, `sk-ant-…` is detected and explicitly rejected with a clear message). The model picker in the UI groups models by provider so the user can match key to model without thinking. The retry, fallback, and cost-estimation paths all branch on provider rather than special-casing each model. Adding a third provider in the future is now a one-block edit in `_make_llm` and one row in `MODEL_PRICES`.

FastAPI lifespan over `@app.on_event`. The original code used `@app.on_event("startup")` to log the OCR availability line once at server boot. Starlette has been emitting a `DeprecationWarning` for that decorator since the 0.93 series; on Python 3.14 the warning shows up in the startup log and looks alarming even though nothing is broken. v3 migrates to the `lifespan` async context manager pattern, which is the documented replacement and yields a clean startup log.

---

3. How It Works

The app is a FastAPI service. The repository is intentionally flat:

- `main.py` — FastAPI app, WebSocket pipeline, the two mode branches (Fast / Full), the provider abstraction, retry-with-backoff, the structured-output recovery chain, and the cache write.
- `agents.py` — the LangGraph state machine, all node functions, the tool-execution loop.

- `knowledge_base.py` — the 34 risk-pattern corpus and the TF-IDF retriever.
- `tools.py` — the LangChain `@tool` wrappers.
- `database.py` — SQLAlchemy models with SQLite WAL-mode pragmas.
- `templates/index.html` — single-page client (setup / live analysis / verdict dashboard / past-reports drawer).
- `tests/test_main.py` — 33 unit tests.
- `Dockerfile`, `requirements.txt`, `LICENSE`, `samples/sample_contract.pdf`, `samples/contract_balanced.pdf`, `samples/contract_mixed.pdf`.

3.1 The two modes

Before each run the user picks a mode in the setup view.

Fast mode runs three sequential LLM calls directly from the PDF — Alex synthesises the bullish case, Sam attacks it, Maya rules. That is the entire pipeline. 3 API requests per analysis. About 6 runs per day on the Gemini free tier (20 requests per day on flash).

Full mode runs the LangGraph multi-agent pipeline described below. 8 to 15 API requests per analysis depending on which specialists the Planner picks and how often Sam calls the precedent search tool. 1 to 2 runs per day on the Gemini free tier; effectively unmetered on a paid OpenAI key.

Both modes write to the same cache and emit the same verdict payload, so the dashboard does not need to know which mode produced the answer.

3.2 Walk-through of a Full-mode run

A full run goes through six phases.

1. **Planner.** Reads the document and outputs a JSON list of 2–5 specialists from `{financial, legal, data, compliance, operations}` that are relevant to this contract. The planner's job is to spend one cheap call to avoid running specialists that have nothing to say.
2. **Specialists.** The selected specialists run one after another. Each one is bound to the `search_precedent` tool and can call it 1–3 times during its analysis. Each writes a short markdown report with a severity rating.
3. **Alex (Strategist).** Reads every specialist report and synthesises the strongest possible bullish case for the deal.
4. **Sam (Red Team).** Reads the specialist reports and Alex's bullish case. Quotes Alex's points and dismantles them. Sam also has access to `search_precedent` and uses it to ground specific attacks in documented precedent.
5. **Maya (Judge).** Reads everything and emits a markdown rationale followed by exactly one fenced JSON block. The JSON validates against a Pydantic schema with `Literal` constraints on verdict and severity enums.
6. **Critique → optional revision.** Alex and Sam each read Maya's ruling. They respond either `ACCEPT:` or `DISSENT:` followed by a specific point Maya overlooked. The two critiques run in parallel via `asyncio.gather`. If at least one dissents, Maya runs once more with both responses in context and emits a revised ruling. If both accept, the original ruling stands.

The whole sequence streams over a single WebSocket so the browser sees every token as it is produced.

3.3 Provider abstraction

`detect_provider(api_key)` looks at the key prefix and returns `"google"`, `"openai"`, `"anthropic"` (detected but rejected), or `None`. `model_provider(model_id)` looks at the model name and returns the same enum. The WebSocket setup handler cross-checks them and refuses the run with a clear error if they disagree ("the `gpt-5` model belongs to the openai provider, but the key you pasted is a google key"). The fallback ladder for structured-output recovery is also per-provider: Gemini failures escalate to `gemini-2.5-pro`, OpenAI failures escalate to `gpt-4o`. Cost estimation uses a per-model price table; new models are one row.

The OpenAI dependency is optional. If `langchain-openai` is not installed the app still boots for Gemini users and rejects OpenAI keys with an install hint rather than crashing.

3.4 The knowledge base

`knowledge_base.py` ships with 34 hand-curated entries across 15 categories: liability, indemnification, termination, pricing, data, IP, disputes, SLA, assignment, confidentiality, governing law, warranty, exit, compliance, and audit — basically the surface area of a commercial contract.

Retrieval is TF-IDF with cosine similarity, implemented in roughly 60 lines of Python. The corpus is tokenised and vectorised once at module load, so each `search_precedent(query)` call is a single dot-product against 34 sparse vectors. No external embedding service, no vector store, no extra deps.

When the model calls the tool, the top-4 matches come back as a small JSON list with title, pattern, risk, and mitigation. The model then quotes that language verbatim in its analysis, which is how the final risk-matrix mitigations end up grounded in documented precedent rather than invented from training memory.

3.5 The structured-ruling recovery chain

Maya's prompt asks for a fenced JSON block at the end of her response. Most of the time she complies. Sometimes she does not, and "sometimes" was frequent enough during development that I built four fallback layers:

1. A regular expression locates the last fenced ```` ```json ```` block in Maya's text. `json.loads` it. Validate against the Pydantic model with strict `Literal` enums.
2. If that fails, send Maya's full text back to the same model at `temperature=0` with the instruction "convert this narrative into clean JSON, output only the JSON." Parse and validate.
3. If that still fails, retry against the provider's stronger model — `gemini-2.5-pro` for Google, `gpt-4o` for OpenAI. Both are slower and more expensive but almost never get the JSON wrong.

4. If even the strong model fails, a hand-written heuristic scans the text for the words `NO-GO`, `CONDITIONAL`, or `GO` and constructs a default ruling with a synthetic risk score and four explanatory rows.

The first layer succeeded on every real test run I executed. The second and third layers were exercised during development by injecting malformed JSON into fixtures. The fourth layer has never fired on a real run but exists so the app cannot crash on the user.

3.6 The cache key and what it buys

The cache is one SQLite table keyed on `SHA-256(extracted_text + model_name)`. Three things follow from that choice:

- A renamed copy of the same PDF returns a cache hit — the filename is not in the key.
- A different model returns a different key, so switching from `gemini-2.5-flash` to `gpt-5` correctly forces a fresh ruling.
- A one-character change in the contract text gives a totally different hash, so the cache cannot be poisoned by an almost-identical file.

The cache row stores the transcripts, the verdict, the risk score, the structured ruling, the input/output token counts, the list-price cost, and metadata flags (truncated, chunked, dissented). What it does *not* store is the API key.

3.7 The UI

The frontend is one HTML file. Three views plus a side drawer, all driven by a small state machine in JavaScript. Setup view collects the API key, model (grouped by provider), mode, and PDF. Live-analysis view shows three cards (Alex / Sam / Maya) streaming Markdown live as tokens arrive, with the planner and specialist activity in the footer log. Verdict dashboard shows a radial risk gauge, the verdict word in semantic colour (GO → "PROCEED", NO-GO → "WALK AWAY", CONDITIONAL-GO → "MAYBE"), a four-row metrics column (time, tokens, cost, model), the risk matrix, the conditions list (when applicable), and a collapsible transcripts panel. A **Past reports** drawer slides in from the side with cached rulings tagged by verdict, model, token count, and timestamp, so the user can re-open or delete previous runs without re-uploading the PDF.

Ctrl+Enter from setup starts the run. Esc cancels a running analysis or closes the open panel.

---

4. Trial and Error

Six things that did not work on the first try.

4.1 Maya did not always produce clean JSON

The biggest problem in version 1 was that the Judge would not reliably end her answer with a parseable JSON block. Most of the time she did. But often enough to matter, the JSON had extra backticks wrapped around it (a fenced block inside a fenced block), a stray sentence after the closing backticks, two JSON blocks (one as a worked example in the rationale, one as the real answer), or almost-right JSON with a trailing comma, or `"high"` instead of `"High"`, or a missing key.

The first version just crashed on any of these. I learned about it the hard way the first time I demoed the app to myself and saw a 500 error after waiting 30 seconds for the verdict.

The fix is the four-stage recovery in §3.5. Most of the time stage one wins. Stages two through four exist so the app degrades rather than crashes.

4.2 The WebSocket kept burning quota after disconnects

The second problem was that WebSockets do not have a clear lifecycle the way HTTP requests do. If I closed the browser tab while Alex was streaming, the server kept generating tokens into nothing. I only noticed when I left a tab open overnight, came back, and found most of my daily free-tier quota gone.

The fix on the server side was to wrap every `_send(ws, ...)` call in the natural WebSocket error path. When the socket drops, the next send raises `WebSocketDisconnect`, that exception bubbles up through the `astream` loop, and the in-flight Gemini call gets cancelled. On the client side, a `close` event listener resets the UI back to the setup view if the disconnect happened mid-stream. I also added a Cancel button that calls `socket.close(4000, "user-cancel")` so users can explicitly stop a run instead of closing the tab.

4.3 LangGraph would not await my coroutines

When I rebuilt the pipeline on LangGraph, the very first run gave me this:

```
InvalidUpdateError: Expected dict, got <coroutine object _node_planner at 0x...>
```

I had registered each node as a sync lambda returning a coroutine:

```
g.add_node("planner", lambda s: _node_planner(s, llm, emit))
```

This looked fine, but LangGraph inspects each callable to decide whether to await its result. A sync lambda that *returns* a coroutine is not recognised as a coroutine function, so LangGraph passed the unawaited coroutine straight to the StateGraph reducer, which complained it got a coroutine instead of a dict.

The fix was to wrap every node in a real `async def`:

```
def _bind(node_fn):
    async def _wrapped(state):
        return await node_fn(state, llm, emit)
    return _wrapped
```

```
g.add_node("planner", _bind(_node_planner))
```

LangGraph then introspects the wrapped function as a coroutine function and awaits it correctly. The lesson was that with frameworks that depend on type/inspection of callables, "this lambda returns a coroutine" is not the same as "this lambda is async."

4.4 The Full pipeline blew through the free-tier daily cap

The multi-agent pipeline does 8–15 LLM calls per run. The Gemini free tier is 20 requests per day on flash. So one Full run could blow through most of the daily allowance, and the next run would die mid-stream with `RESOURCE_EXHAUSTED: 429`.

This is when I realised I had to ship both versions. The current app has a Fast / Full toggle in the setup view. Fast is the three-call pipeline from version 1 (3 requests per run, about 6 runs per day on free tier) and is the default. Full is the multi-agent pipeline (8–15 requests, 1–2 runs per day on free tier) and is opt-in.

I also rewrote the retry logic. Google's 429 errors include a `retryDelay` field saying how long to wait. The retry code now parses that field and uses the suggested delay instead of a fixed exponential backoff. If the error is specifically the daily-cap exhaustion variant (which retrying will never fix), the code surfaces a clear message suggesting Fast mode, a different model, or waiting for the daily reset rather than spinning through useless retries.

4.5 Tailwind silently dropped a hover colour

A small UI bug that took me longer to find than it should have. The Past Reports drawer renders one card per past run, with a hover border that matches the verdict colour. I wrote it as a template string:

```
card.className = `card-soft p-4 hover:border-${accent}-200`;
```

Where `accent` was one of `"emerald"`, `"amber"`, `"rose"`. Looked fine to me. The hover border never showed up on any card.

I spent a while inspecting the DOM and the Network tab before realising what was happening. Tailwind's JIT compiler scans source files for literal class names. It never sees `hover:border-emerald-200` written out anywhere because I assemble it at runtime, so it never emits the corresponding CSS rule. The browser receives a class that the stylesheet does not define.

The fix is ugly but works:

```
const HOVER_BORDER = {
    'GO':             'hover:border-emerald-300',
    'NO-GO':          'hover:border-rose-300',
    'CONDITIONAL-GO': 'hover:border-amber-300',
};
card.className = `card-soft p-4 ${HOVER_BORDER[verdict]}`;
```

Lesson learned: any framework with a build step that depends on static analysis will not pick up strings you build at runtime.

4.6 Two new things in v3: Python 3.14, and FastAPI deprecations

Two smaller bumps surfaced when I tried the app on a fresh Python 3.14 install while preparing the v3 build.

First, `pip install -r requirements.txt` failed with a long Rust backtrace ending in `Failed building wheel for pydantic-core`. The cause is that `pydantic==2.10.4` did not ship pre-built wheels for CPython 3.14, so pip tried to compile `pydantic-core` from source via `maturin` and `cargo` — and most Windows installs do not have a Rust toolchain. The fix was to bump the requirement to `pydantic>=2.11.0,<3.0`. From 2.11 onwards there are pre-built wheels for 3.14 and the install completes in seconds without touching Rust.

Second, the startup log printed a `DeprecationWarning` from FastAPI:

```
DeprecationWarning: on_event is deprecated, use lifespan event handlers instead.
@app.on_event("startup")
```

The OCR-availability log line was wired up via `@app.on_event("startup")`. The replacement is the `lifespan` async context manager passed to `FastAPI(lifespan=...)`. I migrated the handler; the warning is gone and the boot log is clean again.

Neither of these affected behaviour — but a project that emits a Rust compile error on install or a DeprecationWarning on boot looks unfinished, and unfinished is a bad first impression for a project that is otherwise carefully built.

---

5. Numbers

This section reports what I measured against the live OpenAI and Gemini APIs on the current multi-agent architecture.

5.1 Cross-model, cross-document benchmark

I ran the Full pipeline against the three sample contracts in the repository using four different models. Every row is a first run (no cache hit). All times are wall-clock from the WebSocket frame that delivered the PDF to the frame carrying the `verdict` payload back. Costs are list-price equivalents at the providers' published per-million-token rates.

Document	Model	Time	Tokens	Cost (list)	Verdict	Risk
contract_balanced.pdf	gpt-5	617.03 s	6,704	$0.0941	CONDITIONAL GO	61
contract_mixed.pdf	gpt-5	549.98 s	6,623	$0.0823	CONDITIONAL GO	82

Document	Model	Time	Tokens	Cost (list)	Verdict	Risk
contract_mixed.pdf	gpt-5-mini	253.80 s	6,072	$0.0066	CONDITIONAL-GO	78
contract_balanced.pdf	gpt-4o-mini	100.54 s	4,241	$0.0013	CONDITIONAL-GO	70
sample_contract.pdf	gemini-2.5-flash	132.38 s	4,337	$0.0057	NO-GO	95

Three observations are worth pulling out of the table.

The verdict is robust across model families. Every OpenAI run on `contract_balanced.pdf` and `contract_mixed.pdf` came back CONDITIONAL-GO; the Gemini run on the older, more adversarial `sample_contract.pdf` came back NO-GO. The agents disagreed on the *score* by tens of points across models — 61 vs 70 on the same balanced contract is real variance — but they agreed on the *category*. The structured-ruling design is the reason that variance is visible at all: with a free-form summariser there is no number to compare.

`gpt-5-mini` and `gpt-4o-mini` are wildly cheaper than `gpt-5` for very similar verdicts. The full gpt-5 run on `contract_balanced.pdf` cost $0.0941; the gpt-4o-mini run on the same document cost $0.0013. That is a 72× cost reduction for a verdict that landed in the same band (CONDITIONAL-GO, scores 61 vs 70). For the use case "quick scan to decide if you need to read the contract yourself," mini-tier OpenAI models are the obvious default.

Wall-clock time is dominated by the strongest model. A Full run on `gpt-5` (~10 minutes) is roughly six times slower than the same pipeline on `gpt-4o-mini` (~1.5 minutes). The architecture is the same; the bottleneck is the per-call latency of the model, which compounds across the planner-plus-specialists-plus-tribunal call chain.

5.2 Cache replay

The cache-hit measurement gap I called out in the v2 report is now closed. After the `gpt-5` run on `contract_balanced.pdf` completed, I re-uploaded the same PDF with the same model selected. The verdict came back from the SQLite cache in **1.68 seconds** of wall-clock time. The dashboard correctly displayed:

> **FROM CACHE · 617.03 s · SAVED 617.03 s**

…showing that the cache replay saved the full original execution time. Tokens consumed on the replay: zero. Dollar cost on the replay: $0.00, against the original $0.0941. The same SQLite row served the replay; no provider API was contacted.

A similar replay against a previous `gemini-2.5-flash` run on `sample_contract.pdf` returned in 1.68 s as well, saving the original 30.65 s run. The wall-clock floor on a cache hit is set by the WebSocket roundtrip and the artificial token-streaming delay in `_replay_cached` (a ~3 ms-per-64-char drip that makes the cached output animate in like a fresh run), not by any database work — the SQLite lookup itself is sub-millisecond.

5.3 Non-determinism

The Full pipeline samples at non-zero temperature (`0.2`), and the Planner can pick between two and five specialists, and tool-call queries are model-chosen. The three sources of variance compound. Re-running `sample_contract.pdf` on `gemini-2.5-flash` in Full mode with cache disabled produced NO-GO with risk 95 on the first run and CONDITIONAL-GO with risk 88 on the second. Both runs identified the same five core problems; they disagreed on the recommendation tone (walk away vs. fix and proceed) and on which model output the Judge weighted most heavily.

The cache eliminates this variance for repeat queries on the same document — that is the cache's job. Cold runs on the same document will continue to drift between adjacent verdict bands, which is the expected behaviour for sampled LLM output, not a bug.

5.4 Verdict quality

The five adversarial features I planted in the original test document (`sample_contract.pdf`) are independently surfaced by the agents:

- The Legal specialist flags the 3-month liability cap and the one-sided indemnification.
- The Data specialist flags the perpetual royalty-free licence as a privacy and competitive risk.
- The Operations specialist flags the vague "commercially reasonable efforts" SLA.
- The Compliance specialist flags the missing data-export mechanism on termination.

Maya's mitigation language in the risk matrix uses phrasing that maps almost verbatim onto specific KB entries — for the liability-cap row she writes *"introduce carve-outs for data breaches, IP infringement, gross negligence, and willful misconduct"*, which is essentially the `liability_cap_zero_carveouts` entry in `knowledge_base.py`. The grounding is real, not coincidental.

`contract_balanced.pdf` and `contract_mixed.pdf` are softer documents — the agents return CONDITIONAL-GO with explicit conditions rather than NO-GO. The conditions reference things like Total Financial Commitment caps, data/AI usage limits, indemnity supercaps with insurance, change/deprecation controls, escrow and BCDR — all language the Knowledge Base surfaced via the precedent search, not invented by the model.

The system caught what it was supposed to catch. It did not invent risks that were not there.

5.5 The test suite

In parallel with the end-to-end runs, the automated `tests/test_main.py` suite passed all 33 tests. The suite covers content-hash determinism, JSON-block extraction with last-block precedence, Pydantic schema validation with invalid-severity rejection, heuristic-answer correctness across the three verdict bands, WebSocket setup rejection on missing key and bad model, history and delete endpoint behaviour,

chunker overlap arithmetic, transient-error classification, the cost calculator against the published per-million-token prices, the `/health` endpoint's OCR-availability flag, a seeded full-record fetch through `/api/verdict/{id}`, knowledge-base loading and retrieval relevance, tool registration and invocation, and the LangGraph topology.

---

6. Reflection

The Aegis is a small project that started simple and grew in scope as I figured out what was actually missing.

What worked

The four-stage JSON recovery chain is the part I am most pleased with, because it took the system from "crashes when the model misbehaves" to "always returns *something*". Most production LLM applications I have read about either ignore this problem or pretend it does not exist. Handling it properly was harder than building the pipeline itself.

The content-hash cache is the smallest piece of code that does the most work. About fifteen lines of SQLAlchemy and one SHA-256 hash, and a repeated query against an identical PDF returns from the cache with no API call at all. The v3 measurements (§5.2) confirm what the unit tests already showed: the cache cleanly converts the most expensive run in the project — ten minutes on gpt-5 — into a 1.68-second replay for free. The same idea would apply to almost any LLM application that processes deterministic inputs.

The two-mode design (Fast / Full) feels like a real product decision rather than a hack. It lets the same codebase work on free-tier quota and on paid quota without dumbing down either path.

The provider abstraction in v3 was the smallest cleanly-isolated change in the whole project — two functions (`detect_provider`, `model_provider`), one factory (`_make_llm`), one pricing table, one fallback ladder — and yet it converts the app from a Gemini demo into something a user with any major commercial key can drive. The lesson here was to resist the temptation to special-case OpenAI everywhere in the pipeline; the pipeline never needs to know which provider produced a token.

What I would do differently

If I started over, four changes would go in earlier rather than later.

First, I would set the temperature to 0 from the beginning and accept the slightly drier output. The non-determinism issue I documented in §5.3 was avoidable.

Second, I would store the API key in a session cookie rather than asking for it on every load. Users get annoyed pasting their key repeatedly, even when "we never save it" is explicit on the page.

Third, I would migrate to PostgreSQL early. SQLite is great for a solo demo but the single-writer constraint will bite the moment more than one person uses the app concurrently. The SQLAlchemy abstraction makes this essentially a one-line change, so there is no reason to put it off.

Fourth, I would adopt FastAPI's `lifespan` handler and version-floor my Pydantic dependency from day one rather than discovering both problems on a Python 3.14 install at the end. The cost of the deprecation warning is small; the cost of a build failure on a reviewer's machine is large.

What is genuinely missing

There are things I did not finish that I want to be honest about.

- The `/api/history` and `/api/verdict/{id}` endpoints have no authentication. On a shared deployment, anybody with the URL can read every cached verdict. This blocks any public deployment without an auth layer.
- OCR requires the `tesseract` and `poppler` binaries to be installed on the host machine. On Windows that is a manual download. I documented it but did not bundle it.
- The agents pass state through the LangGraph reducer; they do not call each other directly. A future version could add real inter-agent dialogue (Sam asking the Data specialist a follow-up question, for example) rather than the current state-passing flow.
- The per-token cost shown to the user is computed from a hard-coded price table. If Google or OpenAI changes their prices the number will drift.
- Anthropic keys (`sk-ant-…`) are detected but explicitly rejected. Wiring Claude into the provider abstraction is a half-day's work and would be the obvious next addition.

Closing note

Contract review is a domain where the spread between optimistic and pessimistic readings matters more than the centroid. The Aegis tries to make that spread visible rather than hide it inside a single summary. The journey from version 1 (three sequential calls with role personas) to version 2 (a LangGraph state machine with a planner, specialists, tool use, RAG, and a critique loop) to version 3 (the same machinery driven by either Gemini or OpenAI, with measurable cache replay closing the last empirical gap) is the real story of this project. All three versions ship in the same app because each has a place: Fast when you need an answer cheaply, Full when you can afford the calls and want the deeper analysis, and the cache layer makes the question of which mode you used irrelevant on repeat queries — the answer is in the database, and SQLite returns it in under two seconds.